

VendorFlow

AI Product Image Generator

Developer & Platform Documentation
Version 1.0 · July 2026

Built with
Next.js · MongoDB · Stripe · fal.ai · Gemini AI

hello@productvisuals.ai

Table of Contents

1. Overview	3
2. Architecture	4
3. Tech Stack	5
4. Environment Variables	6
5. Data Storage	8
6. Authentication	9
7. Credits & Billing	10
8. Core Features	12
9. API Reference	14
10. Marketplace Integrations	19
11. AI Models	21
12. User Flows	22
13. Setup & Installation	24

1. Overview

VendorFlow is an AI-powered product image generation platform for ecommerce sellers. It enables vendors on marketplaces like Amazon, Flipkart, and Meesho to upload a single product photo and automatically generate a full set of marketplace-ready images — including multiple angles, lifestyle shots, close-ups, infographics, and premium A+ content banners — without a physical photoshoot.

1.1 Core Problem Solved

Most small and medium sellers lack access to professional product photography. They need multiple images per listing across different marketplaces, each with specific size, background, and style requirements. VendorFlow eliminates the cost and complexity of physical photoshoots by using AI image generation to create these assets from a single reference photo.

1.2 Key Capabilities

- Generate 7–8 product images from one uploaded photo
- Support for Amazon, Flipkart, and Meesho with platform-specific sizing and formatting
- Basic product images (nano-banana AI) and premium A+ / RPD banners (GPT Image 2)
- AI-powered photo analysis — auto-fills product name, brand, material, dimensions, color, and key features
- AI-generated listing copy (titles, bullet points, descriptions, keywords, SEO metadata) for all three marketplaces
- Credit-based billing with GST-compliant Indian rupee pricing via Stripe
- Generation history with re-download capability
- Onboarding tour for first-time users
- Bilingual UI (English / Hindi)

Platform

VendorFlow targets Indian ecommerce sellers, particularly those selling on Amazon India, Flipkart, and Meesho. The billing, language support, and marketplace specifications all reflect this focus.

2. Architecture

2.1 High-Level Overview

VendorFlow is a serverless-first Next.js application (App Router). It is deployed as a single Next.js service that handles both the frontend UI and the backend API routes. No separate backend server is required.

Frontend UI	Next.js 15 (App Router), React 19, TypeScript	Pages, forms, image gallery, user account management
API Routes	Next.js Route Handlers (app/api/*)	Authentication, image generation, listing generation, credits, Stripe webhooks
Database	MongoDB (via @mongodb-adapter / NextAuth)	Users, credits, transaction history, generation history, listings
File Storage	Cloudflare R2 (S3-compatible)	Stores generated images; public URL served via R2 CDN
Image Generation	fal.ai (nano-banana, GPT Image 2)	AI product image generation from reference photos
Photo Analysis	Google Gemini 3.5 Flash (Vision)	Extracts product metadata from uploaded photos
Listing Copy	Google Gemini 3.5 Flash (Text)	Generates marketplace listing text (titles, bullets, descriptions)
Auth	NextAuth.js v5 (Credentials + Google OAuth)	User authentication and session management
Payments	Stripe (Checkout + Webhooks)	Credit pack purchases with Indian INR billing and GST receipts
Styling	CSS Modules (globals.css)	Custom design system — no Tailwind or component library
Animations	GSAP + Motion.dev	UI transitions and entrance animations

2.2 Data Flow

The diagram below shows how a typical image generation request flows through the system:

1. User uploads 1–12 product photos in the Studio UI 2. Optional: User clicks "Analyze Photos" — Gemini Vision extracts product metadata 3. User fills in the listing brief (product name, marketplace, image types, style) 4. On submit, the generate API route charges credits and dispatches generation jobs 5. Reference images are uploaded to fal.storage 6. Basic images → nano-banana model; A+ images → GPT Image 2 model 7. Generated images are uploaded to Cloudflare R2 8. Generation record saved to MongoDB (user history) 9. Base64 image data + R2 URLs returned to client; displayed in the results gallery
--

2.3 Directory Structure

```
image-generation/  
├─ app/           # Next.js App Router  
└─ api/          # API Route Handlers
```


3. Tech Stack

3.1 Frontend

Next.js	latest (App Router)	React framework with server components & route handlers
React	latest	UI library
TypeScript	latest	Type safety across the codebase
CSS Modules	globals.css	Styling — custom design system, no Tailwind
GSAP + @gsap/react	^3.15	Scroll-triggered and entrance animations
Motion	^12.x	Micro-interactions and transitions
driver.js	^1.8	Onboarding tour overlay
JSZip	^3.10	Client-side ZIP generation for bulk downloads

3.2 Backend & Infrastructure

NextAuth.js v5 (beta)	Authentication — Credentials provider + Google OAuth	
MongoDB + @mongodb-adapter	Database — users, credits, generations, listings	
Cloudflare R2 (S3 API)	Object storage — generated image files	
fal.ai client	Image generation — nano-banana (basic) + GPT Image 2 (A+)	
Google Gemini 3.5 Flash	Photo vision analysis + listing text generation	
Stripe	Payments — INR Checkout, webhooks, billing portal	
Resend	Transactional email — purchase receipts	
sharp	^0.35	Server-side image resizing (A+ image post-processing)

3.3 Development Tools

ESLint	Code linting
TypeScript	Static type checking
bcryptjs	Password hashing for credential auth

4. Environment Variables

All configuration is managed via environment variables in a `.env.local` file. Required variables are grouped by service.

Required Setup

Copy `.env.example` to `.env.local` and fill in all values before running the application in production. Never commit `.env` files to version control.

4.1 Database

MONGODB_URI	Full MongoDB connection string (mongodb+srv://... or mongodb://...)
MONGODB_DB_NAME	Name of the MongoDB database (e.g. image-generation)

4.2 Authentication (NextAuth)

AUTH_SECRET	Random secret for NextAuth session encryption (openssl rand -base64 32)
AUTH_URL	Public URL of the app (https://yourdomain.com) — used for OAuth redirects
GOOGLE_CLIENT_ID	Google OAuth Client ID from Google Cloud Console
GOOGLE_CLIENT_SECRET	Google OAuth Client Secret

4.3 Image Generation (fal.ai)

FAL_KEY	fal.ai API key (get one at fal.ai) — used for both nano-banana and GPT Image 2
FAL_IMAGE_MODEL	Optional override for the basic image model (default: fal-ai/nano-banana/edit)

4.4 AI Providers

GEMINI_API_KEY	Google AI Studio API key (get one at aistudio.google.com/apikey) — used for Gemini Vision (photo analysis) and Gemini Flash (listing copy generation)

4.5 Cloud Storage (Cloudflare R2)

R2_ACCOUNT_ID	Cloudflare account ID for R2 API
R2_ACCESS_KEY_ID	R2 API token access key ID
R2_SECRET_ACCESS_KEY	R2 API token secret access key
R2_BUCKET_NAME	R2 bucket name (e.g. generated-images)
R2_PUBLIC_URL	Optional custom public base URL for R2 objects (e.g.

	https://cdn.yourdomain.com) — if not set, uses the default R2.dev public URL
--	--

4.6 Payments (Stripe)

STRIPE_SECRET_KEY	Stripe secret key (sk_live ... or sk_test ...) — backend only
STRIPE_WEBHOOK_SECRET	Webhook signing secret from the Stripe Dashboard (whsec ...)

4.7 Email (Resend)

RESEND_API_KEY	Resend API key for transactional email (receipts, OTPs)
EMAIL_FROM	Sender email address (e.g. VendorFlow <hello@productvisuals.ai>)

5. Data Storage

VendorFlow uses MongoDB as its primary database. There is no ORM — all database operations are performed directly via the official mongodb Node.js driver. The lib/mongodb.ts file exports a singleton connection promise that is reused across all API routes.

5.1 Collections

users (NextAuth)	NextAuth user documents	email, name, image, emailVerified
userCredits	Per-user credit balance	userId, balance, welcomeBonusShown, createdAt, updatedAt
creditTransactions	Immutable audit log of all credit changes	userId, type (grant charge refund), amount, description, generationId, priceInr, planId, createdAt
generations	Each image generation batch	userId, productName, category, marketplaces, background, listings, createdAt
generationImages	Individual images per generation	generationId, type, mimeType, storageKey, url, isAplus, createdAt

5.2 Generation Image Storage (Cloudflare R2)

Generated images are stored in Cloudflare R2, not in MongoDB. This keeps the database lightweight and avoids MongoDB's 16 MB document limit.

Storage key format	<userId>/<generationId>/basic-<index>.png
Example	abc123/550e8400-e29b/basic-0.png
Public URL format	https://<R2_PUBLIC_URL>/<storageKey>
Retention	The most recent 5 generations per user are kept; older ones are automatically deleted from R2 and MongoDB via pruneOldGenerations()

5.3 Image Types & Storage Paths

Basic images (nano-banana)	<userId>/<genId>/basic-<N>.<ext>	Standard product listing images
A+ images (GPT Image 2)	<userId>/<genId>/applus-<N>.png	Premium Amazon/Flipkart A+ banners
Reference uploads (temp)	Temporary fal.storage URLs	Short-lived pre-signed URLs used only during generation; not persisted

6. Authentication

VendorFlow uses NextAuth.js v5 (beta) with two authentication providers:

6.1 Providers

Credentials	Email + Password	Passwords hashed with bcryptjs. Registration via <code>/api/auth/register</code> creates the user and then signs in automatically.
Google OAuth	Social Login	Standard OAuth 2.0. User is created on first sign-in via the <code>events.createUser</code> callback.

6.2 User Creation Flow

1. User submits email + password on `/login`
`POST /api/auth/register` — creates user in MongoDB
2. On success, auto-sign-in via `signIn('credentials', ...)`
3. NextAuth `events.createUser` callback fires — grants welcome bonus (1 free credit) and shows popup on first login
4. Session cookie issued; user redirected to `/studio`

6.3 Route Protection

All `/api/*` routes (except `/api/auth/*`) and the `/studio` and `/credits` pages are protected. The `auth()` function from `lib/auth.ts` is called at the top of every route handler. Unauthenticated requests receive a 401 response.

```
const session = await auth();
if (!session?.user?.id) {
  return NextResponse.json({ error: 'Sign in required.' }, { status: 401 });
}
```

6.4 Welcome Bonus

On first registration, a 1-credit welcome bonus is granted atomically using MongoDB's upsert with `$setOnInsert`. This ensures the bonus is granted exactly once — even if the user registers twice or triggers the callback multiple times.

7. Credits & Billing

7.1 Credit System Overview

Credits are the sole currency of VendorFlow. They are deducted before generation begins and refunded if generation fails. Credits never expire.

Pricing

1 basic image = 1 credit | 1 A+ image = 5 credits | Credits are GST-inclusive (18% Indian GST included in plan prices)

7.2 Image Cost Matrix

Basic product images (e.g. Main Product, Front/Side Angle, Lifestyle)	nano-banana (fal.ai)	1 Credit
A+ listing images (Amazon Standard Banner, Three Image Module, etc.)	GPT Image 2 (fal.ai)	5 Credits
Flipkart RPD banners (Hero Banner, Feature Banners, Infographics, etc.)	GPT Image 2 (fal.ai)	5 Credits

7.3 Credit Pack Plans

Starter	₹400	20	—
Growth (Popular)	₹1,000	55	+5 bonus
Pro	₹2,000	115	+15 bonus
Business	₹5,000	300	+50 bonus

7.4 Atomic Credit Operations

Credit charges and refunds use MongoDB's findOneAndUpdate with atomic guards to prevent race conditions:

- chargeUserCredits — deducts credits only if balance >= required amount in a single atomic operation
- addUserCredits — increments balance and is guarded by a Stripe Checkout session ID (idempotency key) to prevent double-crediting on webhook retries
- refundUserCredits — restores credits when generation fails after charging

7.5 Stripe Payment Flow

1. User selects a plan on /credits and clicks Buy
2. POST /api/stripe/checkout — creates a Stripe Checkout session with metadata (userId, plan, credits, price)
3. User completes payment on Stripe's hosted checkout page (INR currency)
4. Stripe sends a checkout.session.completed webhook to /api/stripe/webhook
5. Webhook handler: verifies signature → adds credits via addUserCredits (idempotent by session ID) → sends receipt email via Resend
6. User is redirected to /credits?success=true and sees updated balance

7.6 GST / Invoice Handling

All displayed plan prices include 18% GST. The GST is split into CGST + SGST components for the digital receipt (via `splitGstInclusive()` in `lib/pricing.ts`). Receipts are emailed via Resend after successful payment and can also be viewed as an in-app modal on the `/credits` page.

8. Core Features

8.1 Studio — Image Generation Form

The /studio page is the primary user workspace. It contains a multi-section form for building a generation brief:

Product Photos	Up to 12 images (drag & drop + click-to-upload)	Auto-detected by the AI analyzer on click
Logo (optional)	Single image	Overlaid on A+ banner images
Product Basics	Product name, Category, Brand name, SKU	Required fields
Marketplace	Amazon / Flipkart / Meesho (single select)	Changes the image type options shown
Background	Pure White, Transparent, Lifestyle, Gradient, Custom Color	Affects AI prompt for all generated images
Features, Color & Specs	Key features (chips), product colors (chips), material, dimensions, weight	Auto-filled from photo analysis
Brand Look & Feel	Brand colors (chips), font style	Used for A+ image text overlays
Custom Prompt	Free-text textarea	Appended to all AI prompts; overrides defaults
Basic Images	Checkbox grid of 9 image types	nano-banana — 1 credit each
A+ Images	Checkbox grid of 5 module types	GPT Image 2 — 5 credits each; Amazon only
Flipkart Gallery	8 gallery slots + 11 RPD module types	Gallery = nano-banana; RPD = GPT Image 2
Meesho Gallery	8 gallery + 8 infographic types	All nano-banana at basic pricing
Style Preferences	AI style, text overlay option, language	Applied across all images
Output Settings	Variant sets, quantity per set, format (PNG/JPG/WebP), resolution	Size presets + custom dimensions for basic images
Optional Listing Details	Description, target audience, pricing, warranty, COO, package contents	Used by the listing generation AI

8.2 Photo Analysis (Gemini Vision)

Clicking 'Analyze Photos' sends all uploaded images to the Gemini 3.5 Flash Vision API, which extracts:

- Product type, brand, material, color(s), pattern
- Estimated dimensions and weight
- Target audience and gender
- Key features (up to 6)
- Packaging type, accessories, country of origin, warranty
- Confidence scores per field (high/medium/low)

Extracted fields are auto-populated into the listing brief form, and a toast notification confirms which fields were filled.

8.3 Image Generation (fal.ai)

Two AI models are used depending on the image type selected:

nano-banana/edit	fal.ai	Basic product images (main shot, angles, lifestyle,	4
------------------	--------	---	---

		infographics)	
gpt-image-2/edit	fal.ai	A+ / RPD premium banners (GPT Image 2)	4

The generation pipeline processes images concurrently (up to 4 at a time). Reference images are uploaded to fal.storage first, then used as input URLs in the model calls.

8.4 Listing Text Generation (Gemini Flash)

Clicking 'Generate Listing Text' sends the full brief (product metadata + user inputs) to Gemini 3.5 Flash with a marketplace-specific system prompt. Separate listings are generated for Amazon, Flipkart, and Meesho simultaneously via Promise.allSettled.

- Amazon: Title (≤ 200 chars), 5 bullet points (≤ 500 chars each), description (≤ 2000 chars), SEO title (≤ 50 chars), SEO description (≤ 160 chars), backend keywords
- Flipkart: Title (≤ 100 chars), 5–8 highlights, description, SEO title, SEO description
- Meesho: Title (≤ 80 chars), persuasive Hinglish description, highlights — optimized for value-conscious Tier 2–4 buyers

8.5 Generation History

Every completed generation is saved to MongoDB and appears in the history sidebar. Clicking a history item reloads its images and (if available) its listing text. Old generations beyond the 5 most recent are automatically pruned, with images deleted from R2.

8.6 Download & Export

- Single image — click the download button on any image tile
- All images as ZIP — click 'Download ZIP' to bundle all images
- Listing CSV — export Amazon TSV, Flipkart TSV, or Meesho TSV for bulk upload tools
- Listing JSON — export all listing data as structured JSON

9. API Reference

9.1 POST /api/generate

Generates a batch of product images. Requires authentication.

```
POST /api/generate
Content-Type: multipart/form-data
```

```
Fields:
  brief      - JSON string of GenerationBrief object
  photos     - Image File(s) (up to 12)
```

GenerationBrief object:

productName	string — required
category	string — required
marketplaces	string[] — required
imageTypes	string[] — required, max 12
brandName	string (optional)
sku	string (optional)
background	string (default: 'Pure White')
customBackgroundColor	string (hex, e.g. #FF5733)
material	string (optional)
dimensions	string (optional)
weight	string (optional)
aiStyle	string (default: 'Minimal')
textOnImage	string (default: 'No Text')
language	string (default: 'English')
imageFormat	string (default: 'PNG')
resolution	string ('Standard' 'HD' '4K')
customWidth	number (optional, px)
customHeight	number (optional, px)
keyFeatures	string[]
productColors	string[]
brandColors	string[]
customPrompt	string — appended to all AI prompts

Success response (200):

```
{ "id": "<mongodb_id>", "images": [...], "generationId": "<uuid>", "creditsUsed": <N>,
  "newBalance": <N> }
```

Error responses:

401	Not authenticated
400	Missing or invalid brief / photos
402	Insufficient credits — returns required amount
503	fal.ai not configured
502	AI generation failed — credits are refunded

9.2 POST /api/analyze

Analyzes uploaded photos with Gemini Vision to extract product metadata. Requires authentication.

POST /api/analyze

Content-Type: multipart/form-data

Fields:

photos — Image File(s), max 8

Success response (200) — ProductAnalysis object:

```
{
  "productType": "string",
  "brand": "string",
  "color": "string",
  "material": "string",
  "dimensions": "string",
  "weight": "string",
  "targetAudience": "string",
  "keyFeatures": ["string", ...],
  "countryOfOrigin": "string",
  "warranty": "string",
  "confidence": { "<field>": "high|medium|low", ... }
}
```

9.3 POST /api/listing

Generates AI-written marketplace listing copy for Amazon, Flipkart, and Meesho. Requires authentication.

POST /api/listing

Content-Type: application/json

```
{
  "brief": { ...GenerationBrief },
  "generationId": "<optional — attaches listing to existing generation>"
}
```

Success response (200) — AllListings object:

```
{
  "amazon": { "title", "bullets", "description", "keywords", "seoTitle", "seoDescription" },
  "flipkart": { "title", "highlights", "description", "seoTitle", "seoDescription" },
  "meesho": { "title", "description", "highlights" },
  "id": "<generationId>"
}
```

9.4 GET /api/credits

Returns the authenticated user's credit balance and recent transaction history.

GET /api/credits

Response:

```
{ "balance": <N>, "pricing": { "basic": 1, "aplus": 5 }, "transactions": [...] }
```

9.5 GET /api/credits/welcome

Checks whether the welcome bonus popup should be shown (fires once per user).

```
GET /api/credits/welcome
```

```
Response: { "show": true | false }
```

9.6 GET /api/history

Returns a list of the user's recent generation summaries (without image data).

```
GET /api/history
```

```
Response: [{ id, productName, category, marketplaces, createdAt, thumbnail, imageCount, hasListing }, ...]
```

9.7 GET /api/history/[id]

Returns a single generation with full image URLs and listing text.

```
GET /api/history/[id]
```

```
{ id, productName, category, marketplaces, createdAt, images: [...], listings: {...} }
```

9.8 POST /api/stripe/checkout

Creates a Stripe Checkout session for purchasing a credit pack. Requires authentication.

```
POST /api/stripe/checkout
```

```
Content-Type: application/json
```

```
{ "plan": "starter" | "growth" | "pro" | "business" }
```

```
Response: { "url": "<stripe_checkout_url>" }
```

9.9 POST /api/stripe/webhook

Handles Stripe checkout.session.completed events. Idempotent via session ID. Sends a receipt email via Resend.

```
POST /api/stripe/webhook
```

```
Stripe-Signature: <header>
```

```
// Internal: adds credits, sends receipt, marks session as processed
```

9.10 POST /api/stripe/portal

Creates a Stripe billing portal session for managing payment methods and invoices. Requires authentication.

```
POST /api/stripe/portal
```


10. Marketplace Integrations

10.1 Amazon India

Amazon is the most image-format-diverse integration, with separate pipelines for basic product listing images and premium A+ Content.

Basic Product Images — nano-banana

Main Product (White Background)	1:1 (1024×1024)	Required hero image — pure white BG
Front/Side Angle	3:2	Alternate product view
Key Features Infographic	1:1	Text overlay with key selling points
Dimensions	3:2	Show size/scale for buyer confidence
Product in Use (Lifestyle)	3:2	Lifestyle context shot
Close-up / Material Quality	1:1	Texture and build quality detail
What's in the Box	3:2	Packaging contents
Comparison / Benefits	3:2	Side-by-side or benefit callout
Brand Story / Warranty	2:3 (portrait)	Brand and warranty information

A+ Content Images — GPT Image 2

Standard Banner	970×300	5
Banner with Text Overlay	970×300	5
Standard Image & Text	1000×1000	5
Three Image Module	1464×400	5
Four Image Module	1464×800	5

GPT Image 2 requires sizes that are multiples of 16 with total pixels between 655,360 and 8,294,400. The app requests the closest compliant size and crops to the exact Amazon spec using sharp after generation.

Amazon Export Format

Amazon listings are exported as a tab-separated values (TSV) file with a UTF-8 BOM, ready for direct upload to Amazon Seller Central. Includes all standard catalog fields: SKU, brand, product name, description, 5 bullet points, search terms, dimensions, weight, COO, and warranty.

10.2 Flipkart

Flipkart's integration uses a two-tier image approach: a standard product gallery (1:1 square format) and a Rich Product Description (RPD) section with banner-style infographics.

Flipkart Product Gallery — nano-banana

Hero Image	1024×1024	Primary listing image — white

		background
Front/Alternate Angle	1024×1024	Additional product view
Side View, Back View	1024×1024	Full product coverage
Close-up / Macro	1024×1024	Detail and quality shots
Lifestyle Image	1024×1024	In-context usage shot
Dimension Image, Package/What's Included	1024×1024	Specifications and packaging

Flipkart RPD Modules — GPT Image 2

Hero Banner	1440×600	5
Feature Banner (×4)	1200×600	5 each
Lifestyle Banner	1200×600	5
Infographic	1200×1200	5
Dimensions Graphic	1200×1200	5
Comparison Chart	1200×1200	5
Brand Story	1440×600	5
FAQ Graphic	1200×1200	5
Warranty / Trust	1200×1200	5

Flipkart Export Format

Flipkart listings are exported as TSV with: product name, description, brand, category, MRP, selling price, highlights, stock quantity, manufacturer details, COO, warranty, weight, and dimensions.

10.3 Meesho

Meesho targets value-conscious buyers in Tier 2–4 Indian cities. Listings use a Hinglish style and are optimized for social-commerce discovery. Meesho has no A+ content section — all informational graphics are part of the standard product gallery.

Meesho Product Gallery — nano-banana

Hero Image, Front View, Back View, Side View	1000×1000	Core product views — white/light background
Close-up, Lifestyle, Size/Dimension, Package/What's Included	1000×1000	Detail and usage context

Meesho Infographics — nano-banana (same pricing)

Feature Highlights, Material Details, Fabric Composition, Size Chart	Core informational graphics
Usage Instructions, Wash Care, Color Variants, Package Contents	Value-add images for buyer confidence

Meesho Export Format

Meesho listings are exported as TSV with: title (≤80 chars), persuasive Hinglish description, category, MRP, selling price, discount %, stock, COO, and highlights. The title is optimized for the Meesho search algorithm and typically includes a price indicator.

11. AI Models

11.1 fal.ai — Image Generation

fal-ai/nano-banana/edit	Basic product listing images	4
fal-ai/gpt-image-2/edit	Premium A+ / RPD banner images	4

Both models use the edit (image-to-image) mode, taking a reference photo and an AI prompt as input. Reference images are uploaded to fal.storage and passed as URLs.

11.2 Google Gemini 3.5 Flash

Vision (multimodal)	generativelanguage.googleapis.com	Photo analysis — extracts product metadata from uploaded photos
Text	generativelanguage.googleapis.com	Listing copy generation — titles, bullets, descriptions, keywords, SEO metadata

11.3 Prompt Engineering

The lib/prompts.ts file contains buildImagePrompt() and buildAplusPrompt() which construct detailed, structured prompts from the user-supplied brief. Prompts include:

- Product name, brand, category, and target marketplace
- Background type (white, lifestyle, gradient, etc.)
- Key features and color information
- Target audience and use context
- Output format and aspect ratio
- Style guidance (Minimal, Premium, Luxury, Modern, Colorful, Professional)
- Text overlay instructions (from the 'Text on image' preference)
- Custom user prompt (appended verbatim to allow full overrides)

12. User Flows

12.1 First-Time User Journey

1	Visit site → click Sign Up	Navigate to /login with ? mode=register
2	Enter email + password → Create Account	User created in MongoDB; welcome bonus (1 credit) granted; auto-signed in
3	Welcome modal appears	CreditWelcomeModal shows the 1 free credit bonus; dismissable
4	Redirected to /studio	Onboarding tour highlights the upload zone, brief form, and image type selector
5	Upload 1 product photo	Drop zone or click-to-upload; max 12 images
6 (optional)	Click Analyze Photos	Gemini Vision extracts product type, brand, material, color, features; form auto-fills
7	Fill product name, category, marketplace	Required fields — others pre-filled from step 6
8	Select image types (checkboxes)	Basic images (1 credit each) or A+ images (5 credits each)
9	Click Generate Images	Credits deducted; images generated concurrently via fal.ai; saved to R2 and MongoDB
10	Images appear in gallery	Lightbox on click; download ZIP or individual images

12.2 Credit Purchase Flow

1	Navigate to /credits	Shows current balance, plan cards, transaction history
2	Click Buy on a plan (e.g. Growth — ₹1,000 for 55 credits + 5 bonus)	POST /api/stripe/checkout with plan ID
3	Redirected to Stripe Checkout (INR)	Stripe handles PCI compliance; user enters card details
4	Payment confirmed on Stripe	checkout.session.completed webhook fires
5	Webhook: credits added to account	Idempotent — safe against webhook retries
6	Redirect to /credits?success=true	Balance updates; success banner shown; receipt email sent
7	Optional: click View Bill	ReceiptModal shows GST breakdown (base amount + CGST + SGST)

12.3 Generate → Export Flow

1	Generate images with full brief	All marketplace selections and image types included
2	Click Generate Listing Text	Sends brief to Gemini Flash; generates

		Amazon + Flipkart + Meesho listings simultaneously
3	Listing panel opens with tabbed view	Switch between Amazon / Flipkart / Meesho tabs
4	Copy individual fields or copy all	Click Copy on any field, or 'Copy all' on a section
5	Click Export: Amazon CSV	Downloads Amazon TSV (UTF-8 BOM) for Seller Central
6	Click Export: Flipkart CSV	Downloads Flipkart TSV for Flipkart seller panel
7	Click Export: Meesho CSV	Downloads Meesho TSV for Meesho supplier panel
8	Click Export: JSON	Downloads all listing data as structured JSON for API integrations

13. Setup & Installation

13.1 Prerequisites

- Node.js 18+ (LTS recommended)
- npm or yarn
- MongoDB instance (local or Atlas)
- Cloudflare account with R2 enabled + API token
- fal.ai account + API key
- Google Cloud project + AI Studio API key
- Stripe account (test or live mode)
- Resend account (for email receipts)

13.2 Installation Steps

1. Clone the repository and install dependencies

```
git clone <repo-url>
cd image-generation
npm install
```

2. Configure environment variables

```
cp .env.example .env.local
# Edit .env.local with all required values (see Section 4)
```

3. Start the development server

```
npm run dev
# Open http://localhost:3000
```

4. Build for production

```
npm run build
npm start
```

13.3 Required Service Sign-Up

MongoDB Atlas	mongodb.com/atlas	Connection URI (mongodb+srv://...)
Cloudflare R2	dash.cloudflare.com/r2	Account ID, Access Key, Secret, Bucket Name
fal.ai	fal.ai	API Key (FAL_KEY)
Google AI Studio	aistudio.google.com/apikey	API Key (GEMINI_API_KEY) — starts with AIza...
Stripe	dashboard.stripe.com	Secret Key + Webhook Secret
Resend	resend.com	API Key
Google OAuth	console.cloud.google.com	OAuth Client ID + Secret

13.4 Stripe Webhook Setup (Local Testing)

To test Stripe webhooks locally, use the Stripe CLI:

```
stripe listen --forward-to localhost:3000/api/stripe/webhook
# Copy the webhook signing secret (whsec_...) output by this command
# Paste it as STRIPE_WEBHOOK_SECRET in .env.local
```

13.5 Deployment

VendorFlow is a standard Next.js application deployable to any platform that supports Node.js:

Vercel	Recommended — zero-config Next.js deployment; add env vars in dashboard
Railway	Add MONGODB_URI, R2, Stripe, fal, Gemini vars as environment variables
Render	Use a Dockerfile or build command: npm run build; start command: npm start
Self-hosted (VPS)	Run npm run build && npm start behind a reverse proxy (nginx/Caddy)

13.6 Environment Variable Checklist

MONGODB_URI	Yes	All API routes	MongoDB Atlas dashboard
MONGODB_DB_NAME	Yes	lib/*.ts	Your choice — must match a database on the cluster
AUTH_SECRET	Yes	NextAuth	openssl rand -base64 32
AUTH_URL	Yes	NextAuth	Your deployed URL (https://domain.com)
GOOGLE_CLIENT_ID	Yes (OAuth)	NextAuth	Google Cloud Console → Credentials
GOOGLE_CLIENT_SECRET	Yes (OAuth)	NextAuth	Google Cloud Console → Credentials
FAL_KEY	Yes	lib/fal.ts	fal.ai dashboard
GEMINI_API_KEY	Yes	lib/analyze.ts, lib/listing.ts	aistudio.google.com/apikey
R2_ACCOUNT_ID	Yes	lib/cloudflare.ts	Cloudflare dashboard → R2 overview
R2_ACCESS_KEY_ID	Yes	lib/cloudflare.ts	Cloudflare API Token
R2_SECRET_ACCESS_KEY	Yes	lib/cloudflare.ts	Cloudflare API Token
R2_BUCKET_NAME	Yes	lib/cloudflare.ts	Cloudflare R2 → bucket name
STRIPE_SECRET_KEY	Yes	lib/stripe.ts, webhook	Stripe dashboard → Developers → API keys
STRIPE_WEBHOOK_SECRET	Yes (webhooks)	webhook route	stripe listen CLI output
RESEND_API_KEY	No (recommended)	lib/email.ts	resend.com → API Keys
EMAIL_FROM	No (recommended)	lib/email.ts	Your verified domain in Resend

